UD1. B – Persistència en Fitxers Json



Mòdul: Accés a Dades

1.Què és JSON?

JSON és un format d'intercanvi de dades que es basa en la sintaxi d'objectes de JavaScript. Està compost per parells de clau-valor i s' utilitza per representar dades de manera estructurada i fàcilment llegible.

2.Accés a fitxer JSON amb la llibrería JSON.simple

És una biblioteca simple de Java per al processament JSON, lectura i escriptura de dades JSON i compliment tota lEspecificació JSON (RFC4627)

Dependència JSON-simple.

pom.xml

<dependency>
<groupId>com.googlecode.json-simple</groupId>
<artifactId>json-simple</artifactId>
<versió>1.1.1</versió>
</dependència>

Parsear

JSONParse

Proporciona accés directe de només lectura a les dades JSON de forma de streaming. Aquesta és la forma més eficaç de llegir dades JSON.

La classe Json conté mètodes per crear analitzadors a partir d'orígens d'entrada (InputStream i Reader).

JSONObject:

```
{"key":"value","key":"value","key":"value"}
```

Para obtener determinado valor en determinada clave usamos:

```
objetoJson.get("clave")
```

JSONArray：

```
[{"key":"value","key2":"value2"},{"key":"value","key2":"value2"},
{"key":"value","key2":"value2"}]
```

La diferència és que JSONObject és un objecte (Object), que correspon amb una estructura de Document embolicat en { }, mentre que JSONArray és una matriu (Array) embolicat en [ ]. JSONArray està compost per un o més JSONObjec.

LLegir JSON d'un arxiu

Llegir un l'arxiu JSON per eixemple "test.json" generat anteriorment i mostra cada valor.

Test.json

```
{"name":"mkyong2.com",
"messages":["msg 1","msg 2","msg 3"],
"age":29}
```

JsonSimpleReadExample.java

```
package FichJson;

import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.json.simple.parser.ParseException;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Iterator;


public class JsonSimpleReadExample {
 public static void main(String[] args) {
 JSONParser parser = new JSONParser();
 try {
 Object obj = parser.parse(new FileReader("test.json"));
 JSONObject jsonObject = (JSONObject) obj;
 System.out.println(jsonObject);
 String name = (String) jsonObject.get("name");
 System.out.println(name);
 long age = (Long) jsonObject.get("age");
```

```
System.out.println(age);
// loop array
JSONArray msg = (JSONArray) jsonObject.get("messages");
Iterator<String> iterator = msg.iterator();
while (iterator.hasNext()) {
System.out.println(iterator.next());
}
} catch (FileNotFoundException e) {
e.printStackTrace();
} catch (IOException e) {
e.printStackTrace();
} catch (ParseException e) {
e.printStackTrace();
}
```

LLegir un document JSON amb un array de documents

Test2.json

```
{
"id": "0912898213",
"pages": [{"pageUrl": http://www.example0.com,
             "results": [{"number": "1"},{"number": "2"}]
             },
      {"pageUrl": "http://www.example1.com",
             "results": [{"number": "3"},{"number": "4"}]}
      ]
}
```

FichArray.java

```
package FichJson;
import java.io.FileReader;
import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
```

```
public class FichArray {

public static void main(String[] args) throws Exception {
JSONParser parser = new JSONParser();
JSONObject pagesObject=(JSONObject) parser.parse(new FileReader("test2.json"));
System.out.println(pagesObject.get("id"));
// pages es un JSPNArray
System.out.println(pagesObject.get("pages").getClass().getName());
//Obtereel array
JSONArray jsonArray= (JSONArray) pagesObject.get("pages");
//recorer el arrat y obtiene el documento
for(int i=0; i<jsonArray.size(); i++){
JSONObject jo = (JSONObject) jsonArray.get(i);
System.out.println(jo.get("pageUrl"));
 JSONArray jsonArraydoc= (JSONArray) jo.get("results");
 for(int j=0; j<jsonArraydoc.size(); j++){
      JSONObject jo2 = (JSONObject) jsonArraydoc.get(j);
System.out.println(jo2.get("number"));}
// System.out.println(jsonArray.get(i));
}
}
}
```

Escriure JSON en un archivo

Escriure dades passa **JSONObject** escrivint dades JSONJSONArray i guárdelo en un arxiu

JSonSimpleWrite.java

```
JsonSimpleWriteExample

import java.io.FileWriter;
import java.io.IOException;
// TODO Auto-generated method stub
import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
```

```
public class JsonSimpleWriteExample {
 public static void main(String[] args) {
 JSONObject obj = new JSONObject();
 obj.put("name", "mkyong2.com");
 obj.put("age", 29);
 JSONArray list = new JSONArray();
 list.add("msg 1");
 list.add("msg 2");
 list.add("msg 3");
 obj.put("messages", list);
//Fichero escritura, agregar
 try (FileWriter file = new FileWriter("test.json", true)) {
 file.write(obj.toJSONString());
 file.flush();
 } catch (IOException e) {
 e.printStackTrace();
 }
 System.out.print(obj);
 }
```

3.Serialitzar i deserialitzar JSON a Java 8

La serialització i deserialització de JSON són processos que permeten convertir objectes Java en format JSON i viceversa. JSON (JavaScript Object Notation) és un format d'intercanvi de dades lleuger i fàcil de llegir i escriure. En serialitzar un objecte, aquest es converteix en una cadena de text JSON. En deserialitzar, la cadena de text JSON es converteix novament en un objecte Java.

Librerías de JSON

En Java 8, existen varias librerías populares que puedes utilizar para trabajar con JSON, como:

- Jackson
- Gson
- JSON-B

Serialización de JSON

Per serialitzar un objecte Java a format JSON a Java 8, pots utilitzar alguna de les llibreries esmentades anteriorment. Aquestes llibreries proporcionen mètodes i anotacions que et permeten convertir objectes Java en JSON de manera senzilla.

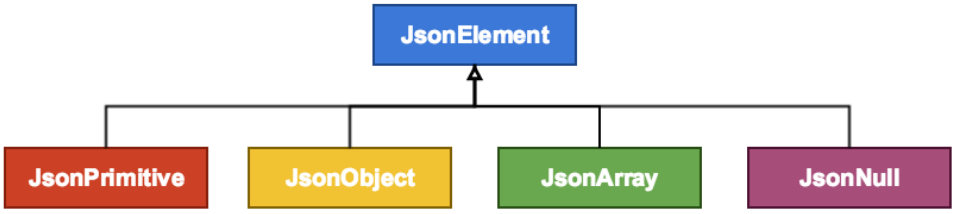
Deserialización de JSON

La deserialització de JSON a Java 8 implica convertir una cadena de text JSON en un objecte Java. Igual que amb la serialització, les llibreries esmentades anteriorment et facilitaran aquesta tasca proporcionant mètodes i anotacions que et permeten convertir JSON en objectes Java.

4.Gson (JSON) en Java

GSON és un API a Java, desenvolupada per Google, que s'utilitza per convertir objectes Java a JSON (serialització) i JSON a objectes Java (deserialització).

- JsonElement: Aquesta classe representa qualsevol element del Json que pot ser d' algun dels següents 4 tipus:
- JsonObject: Aquesta classe representa un objecte al Json; és a dir, un conjunt de parells clau-valor on les claus són strings i els valors són qualsevol altre tipus de JsonElement.
- JsonArray: Aquesta classe representa un arrelament al Json. Un array és una llista de JsonElements cadascun dels quals pot ser d'un tipus diferent. Es tracta d' una llista ordenada, per la qual cosa l' ordre en què s' afegeixen els elements es conserva.
- JsonPrimitive: Aquesta classe representa un tipus de dada primitiva o objectes de dades simples (String, Integer, Double, etc.).
- JsonNull: Representa un objecte a null.



Dependencia pom.xmp llibreries gsonn

```
<dependency>
<groupId>com.google.code.gson</groupId>
<artifactId>gson</artifactId>
<version>2.8.0</version>
</dependency>
```

JsonParser

Analizando más en detalle el código, lo primero hacemos es definirnos un «parser» (JsonParser) para que pasandole un String en formato Json, este nos lo pase a un objeto de algunas de las clases de Gson; en este caso, a un objeto de la clase JsonArray:

JsonArray:

Una vegada que tenim un JsonArray, recorrem aquesta estructura com si d'una llista es tractés, sent cadascun dels elements d'aquesta llista un objecte de tipus JsonElement.

JsonElement

Per definició tots els elements del JSON són de tipus JsonElement, per la qual cosa tot el JSON diem que és un JsonElement.

En primer lloc ens trobem amb un JsonArray que és una llista (o array) d'objectes. Això ho podem reconèixer pel símbol del corxet «[...]». Dins d'aquest JsonArray no trobem amb un únic objecte del tipus JsonObject i aquest objecte al seu torn conté una seríe de JsonElements dels quals 3 són JsonPrimitives i un torna a ser un JsonArray

Dins del «foreach» quan passem cadascun d'aquests elements (recordem que en aquest cas es tracta d'un arrelament amb un únic element) a un objecte de la classe JsonObject i a partir d'aquest objecte obtenim cadascun dels seus atributs amb els mètodes «getAsXXX()»,, depenent del tipus de dades que sigui. Al final contruïm l'objecte Pojo amb els valors de l'objecte JSON

FromJson()

classe «Gson» i amb el mètode «fromJson» passant-li com a parametres l'String amb el Json i la classe POJO, el transforma automàticament a una llista d'objectes de la classe Pojo

toJson()

de la classe «GSON» passant-li com a paràmetre la llista de Pojos et retorna n String en format JSON amb la llista dels Documents JSON: